APRESENTAÇÃO DO PRODUTO · VERSÃO 1.1.1

QA Scout

Explore com missão. Volte com evidência.

Extensão do Chrome, gratuita e de código aberto, para teste exploratório baseado em sessões. Registra o que acontece enquanto você explora e entrega o relatório e o bug prontos para o time.

0
requisições de rede: os dados ficam no seu computador

4
idiomas: pt-BR, pt-PT, inglês e espanhol

176
testes automatizados no próprio projeto

Thamires Azevedo · QA Sênior · github.com/azevedothamirest/qa-scout

O CONCEITO

O que é teste exploratório?

É aprender sobre o sistema, desenhar os testes e executá-los **ao mesmo tempo**. Em vez de seguir um roteiro fechado, quem testa investiga com uma missão, observa o comportamento real e decide o próximo passo pelo que acabou de ver.

Roteiro (script)

Verifica o que já se sabe que deveria acontecer. Bom para regressão e automação.

Exploratório

Descobre o que ninguém pensou em escrever: combinações, bordas, fluxos reais. É onde aparecem os bugs mais caros.

Sessões com tempo marcado (SBTM)

O **Session-Based Test Management**, de Jon e James Bach, dá estrutura sem virar roteiro. Cada sessão tem três partes:

1. **Charter:** a missão, no modelo de Elisabeth Hendrickson.
2. **Sessão:** um bloco de tempo sem interrupção (30 a 90 min), anotando bugs, dúvidas, ideias, riscos e observações.
3. **Debriefing:** revisão das notas, divisão do tempo em Teste, Bug e Setup (TBS) e o que fica para a próxima sessão.

Explorar **o checkout com Pix** com **QR expirado, botão voltar e cupom** para descobrir **pedidos pagos que ficam pendentes**.

Divisão do tempo (TBS)



Teste 60% · Bug 25% · Setup 15%. Muito tempo em setup ou bug indica ambiente instável ou produto frágil: uma métrica que o time inteiro entende.

O PROBLEMA

Exploratório costuma ser trabalho invisível

Encontra os melhores bugs, mas deixa pouco rastro: o que foi explorado, por quanto tempo, com qual evidência? E reproduzir o bug depois vira arqueologia.

A SOLUÇÃO

O QA Scout anota enquanto você explora

📸 Captura automática

Cliques, campos preenchidos e páginas abertas viram notas com print, no painel lateral do Chrome. **O valor digitado nunca é lido.**

🧯 Erros de console e rede

Erro de JavaScript, `console.error`, respostas 4xx/5xx e falhas de conexão viram notas sugeridas como bug ou risco.

🪜 Passos para reproduzir

Um clique monta a lista com as ações que vieram antes do bug.

🐞 Bug pronto pro Jira

"Copiar bug" gera título, ambiente, navegador, página, passos, obtido x esperado e os erros do mesmo minuto.

🃏 Baralho de heurísticas

18 cartas (tours de Whittaker, dados, interação, contexto) para quando a criatividade travar.

📄 Relatório da sessão

Debriefing com TBS e relatório em Markdown ou PDF para colar no Jira, Notion ou Slack.

Você revisa tudo. As notas automáticas chegam "a revisar": confirme, edite ou exclua. Só as revisadas entram no relatório.

PARA O TIME INTEIRO

Como cada área aplica

Exploratório não é exclusividade de QA. Com uma missão curta e 20 minutos, qualquer pessoa do time encontra o que o teste roteirizado não pega.

QA

Sessões com missão e evidência

- Planeje a sprint em charters por risco: o que mais pode custar caro?
- Use o TBS no debriefing para mostrar onde o tempo foi parar.
- Abra bugs com passos e erros de console já anexados, sem "não consigo reproduzir".
- Transforme o que a sessão descobriu em casos de regressão automatizada.

Dev

Explorar antes de abrir o PR

- Uma sessão de 15 min na própria feature: "explorar o formulário novo com dados inválidos para descobrir validações faltando".
- Bugs chegam com passos, ambiente, URL e o erro de JavaScript do mesmo minuto: menos ida e volta.
- Na revisão de PR, explore a branch em homologação e cole o relatório no PR.

Produto

Validar a experiência real

- Explore a jornada como cliente antes da release: "explorar o onboarding com um usuário novo para descobrir onde a pessoa trava".
- Notas de **dúvida** e **ideia** viram perguntas de refinamento e backlog.
- O relatório mostra cobertura e riscos em linguagem de negócio, ótimo para decidir o go/no-go.

💡 Ritual sugerido: bug bash de 30 minutos

Cada pessoa (QA, dev, produto, design) pega um charter diferente, todos exploram ao mesmo tempo e o time junta os relatórios no fim. Rápido, barato e revela muito antes da release.

COMEÇAR

Instalação em 1 minuto

🧩 Extensão do Chrome (recomendado)

Instale pela Chrome Web Store, abra o painel lateral pelo ícone da bússola e comece uma sessão. O Chrome pede acesso aos sites no primeiro uso.

🖥️ Navegador de teste (sem mexer no seu Chrome)

Com Node.js 18+:

```
npx github:azevedothamirest/qa-scout https://seu-sistema
```

Passo a passo completo, código-fonte e tutorial: **github.com/azevedothamirest/qa-scout** (veja `docs/tutorial.md`).

COM IA

O Claude explora com você (MCP)

O QA Scout tem um servidor **MCP** que deixa o Claude conduzir a sessão no navegador de teste: ele lê a tela, clica, preenche, consulta os erros capturados, anota o que encontra e devolve o relatório. A sessão fica no histórico para você revisar.

```
claude mcp add qa-scout -- npx -y github:azevedothamirest/qa-scout mcp
```

"Use o QA Scout para uma sessão de 20 minutos em https://homolog.minhaloja.com: explorar o checkout, com cupons e Pix, para descobrir cobranças erradas. Me mostre o relatório."

1. Lê

Entende a tela pela árvore de acessibilidade.

2. Explora

Clica e preenche como uma pessoa; a captura grava tudo.

3. Anota

Registra bugs, riscos e dúvidas e encerra com o relatório.

Roda no terminal (Claude Code ou Claude Desktop). O texto das páginas visitadas vai para o Claude: use só ambientes e dados de teste.

PODE USAR NO TRABALHO?

Privacidade e segurança

- ✓ Sem servidor, sem conta, sem telemetria: **nenhuma requisição de rede** (bloqueada por CSP `connect-src 'none'`).
- ✓ Dados só no armazenamento local do navegador; saem apenas quando você copia ou baixa algo.
- ✓ Valores digitados nunca são lidos; tokens e senhas nas URLs são mascarados.
- ✓ Código aberto (licença MIT): dá para auditar tudo o que ele faz.
- ✓ Material para o time de segurança: `docs/seguranca.md`, termos de uso e `SECURITY.md`.

Resumo

QA

Evidência e rastreabilidade do exploratório.

Dev

Bugs reproduzíveis, com contexto técnico.

Produto

Visão real da experiência e dos riscos.

Explore com missão. Volte com evidência.

github.com/azevedothamirest/qa-scout · Thamires Azevedo · @qa.thami